

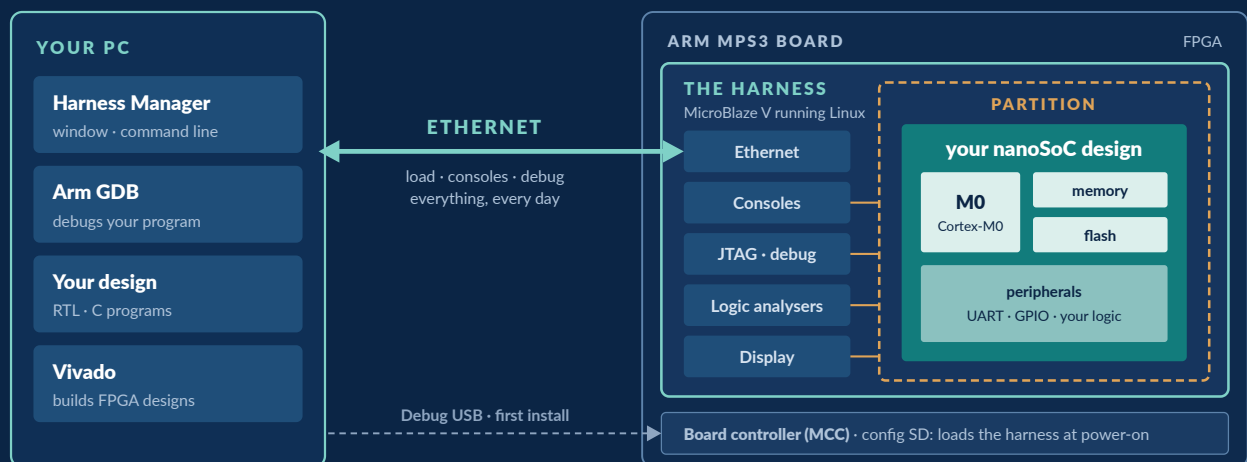
GETTING STARTED

nanoSoC:

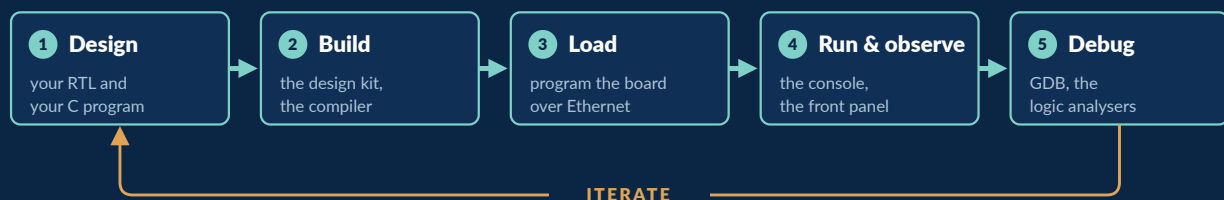
MPS3 Verification Platform

From an Arm MPS3 board in its box to your own program, and your own FPGA design, running and debugged on it over one Ethernet cable.

THE PLATFORM



HOW YOU WORK ON IT



Platform v2.0 · Harness Manager 1.0.0 · 7 October 2026

Companions: the *User Guide* and the *Technical Reference & Maintainers Guide*.

SoC Labs. Arm IP (Cortex-M processors, CoreSight, CMSDK) is used under licence from Arm and is not part of the platform.

Introduction

This guide is for a researcher or teacher who has an Arm MPS3 FPGA board, or will soon, and wants to run and debug their own system-on-chip on it. You need no experience of FPGA boards: each idea is explained when you first need it.

By the end you will have:

- put the platform's harness on your board, and named and claimed it as yours;
- loaded ready-made designs and watched their consoles;
- with Arm Academic Access, built the nanoSoC design once and halted its Cortex-M0 processor in a debugger;
- changed, built and run a C program of your own on that processor, in minutes;
- optionally, changed a small piece of FPGA logic, built it and loaded it.

Read chapters 1 to 4 first: they explain the board, the harness and the words this guide uses. Then work through chapters 5 to 13 in order, once. Chapters 11 to 13 need the nanoSoC source (below); without it, skip them. Chapter 14 is optional; chapter 15 is there for when something goes wrong.

What you need first

- An Arm MPS3 board (V2M-MPS3), its power supply and its Debug USB cable, plus an Ethernet cable.
- A PC running Linux, macOS or Windows, with a free Ethernet port.
- A microSD card of 4 to 32 GB, and a card reader for your PC.
- For the nanoSoC chapters (11 to 13): Arm Academic Access, and AMD Vivado with a licence for the board's FPGA. nanoSoC contains Arm IP, so the download carries no nanoSoC design: you build it once yourself, from the nanoSoC source, which comes from SoC Labs, on proof of your Arm Academic Access (contact SoC Labs support).

Chapter 5 checks each item in detail. Everything else is a free download from <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases>.

How to read the steps

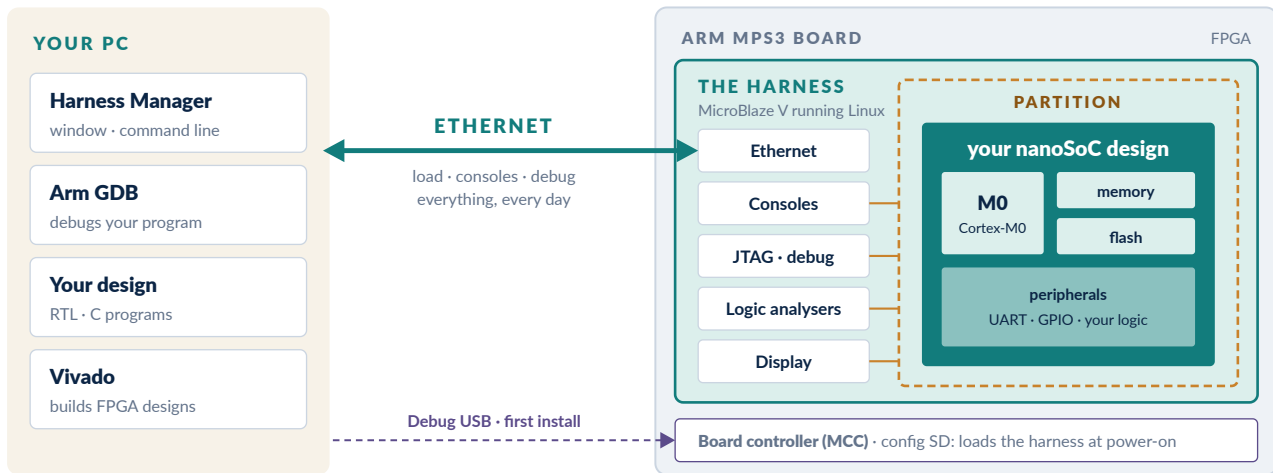
You work in Harness Manager's window. Each step says what to click; a box headed *From the command line* under it gives the same step as commands, for scripts.

YOU SEE	IT MEANS
Workbench → Program	A tab, card or button in the window, in the order you click them.
<code>harness-manager info \$B</code>	A command to type. <code>\$B</code> is your board's address, which you set once per terminal (§6.4).
Grey text under a command	What it prints, from a real session. <code>...</code> marks lines left out.
<code>(gdb) monitor halt</code>	A command typed at the debugger's prompt.
<code><zip></code>	Something you fill in.

For help, contact SoC Labs support (the support contact that came with your board).

The platform at a glance, and why

The platform turns an MPS3 FPGA board into a test bench for system-on-chip designs that you drive from your PC over Ethernet. You load a design in about a minute, read its console and debug its processor, without touching the board.



- **The board.** The Arm MPS3 carries a large FPGA, a chip whose logic you configure to become any digital circuit, including a whole SoC. On its own, the board loads one design from an SD card at power-on and keeps it until the next power-on.
- **The harness** is a permanent design for the FPGA, supplied with the platform. It is a small Linux computer built from FPGA logic, and it looks after the network, the consoles and the debugger. You install it once.
- **The partition** is the part of the FPGA the harness leaves free. It holds the design under test, and the harness rewrites it over Ethernet while it keeps running. *Why:* you change designs in a minute, from your desk, and a design that hangs can never lock you out.
- **nanoSoC** is SoC Labs' family of Arm Cortex-M systems-on-chip, built on an AHB bus fabric, for research, teaching and silicon. The harness is built around it: a nanoSoC design drops into the partition, and its console, debug port and flash reach you through the harness.
- **Harness Manager**, on your PC, is the window and command line you drive it all from. It needs the board's Debug USB cable once, to install the harness; after that, only Ethernet.

Chapters 1 to 4 explain each piece in turn. Then the guide follows the flow on the cover: set up, load a design, run and debug your code, change it, and go round again.

Contents

1	The MPS3 board	5
2	Why a harness	6
3	nanoSoC	7
4	How you'll work	8
5	What you need	10
6	Install your tools	10
7	Put the harness on the board	13
8	Put Linux on the microSD	15
9	Claim and name your board	17
10	Your first designs	19
11	Build the nanoSoC design	21
12	Meet nanoSoC on the board	23
13	Your own program	26
14	Your first FPGA design	29
15	When something goes wrong	32
16	Where next	33
A	Cheat sheet	34
B	Glossary	35

1 The MPS3 board

Before the platform, the board on its own: what is on it, and how it starts. Every later chapter builds on these parts.

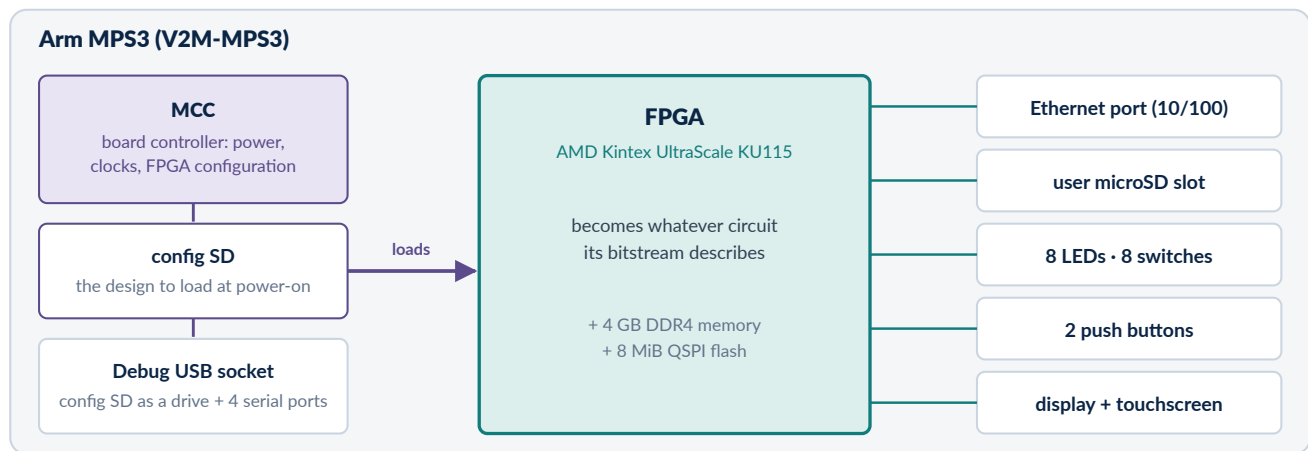


Figure 1.1. The parts of the MPS3 this guide uses. The MCC loads the FPGA from the config SD; everything on the right belongs to the FPGA's design.

1.1 The parts

- **The FPGA** (field-programmable gate array) is the large chip in the middle, under a fan. Its logic is configured by a file called a **bitstream**, so it can become any digital circuit that fits, including a complete SoC. Its memory, flash, Ethernet port, user microSD slot, LEDs, switches and display are all wired to it.
- **The MCC** (motherboard configuration controller) is a small microcontroller beside it. It switches the board on, sets its clocks and loads the FPGA. You do not program it.
- **The config SD** is the MCC's microSD card. It holds a few text files and the bitstream the MCC loads at power-on.
- **The Debug USB** socket connects the board to a PC. Through one cable the PC sees the config SD as a drive named `V2M-MPS3`, and gets four serial ports: one is the MCC's command console.
- **The user microSD** slot is a second, separate card slot. It belongs to the FPGA, not to the MCC.
- **The front panel** is a small colour display (320 × 240) with a touchscreen. What it shows depends on the FPGA's design.
- **LEDs, switches and push buttons:** eight user LEDs, eight DIP switches and two user push buttons, all wired to the FPGA.

1.2 How the board starts by itself

- 1 **You switch it on.** The MCC starts first.
- 2 **The MCC reads the config SD:** a settings file, then the name of the bitstream for this board.
- 3 **It loads that bitstream into the FPGA** and sets the clocks. The whole FPGA is configured at once.
- 4 **The FPGA runs that design** until the board is switched off or the MCC is told to reboot.

So, on its own, the board runs one design per power-on. To change it you copy a new bitstream onto the config SD over the Debug USB and reboot the board, and whatever was running stops. The next chapter explains why that is slow for day-to-day work, and what the harness changes.

2 Why a harness

The board on its own makes every change slow and hands-on. The harness keeps part of the FPGA running permanently, so that changing and debugging your design becomes a one-minute job over the network.

2.1 The problem

Working with the board on its own, as chapter 1 describes, means:

- **A full rebuild for every change.** The bitstream describes the whole FPGA, so every change to your SoC means rebuilding all of it with AMD's Vivado tools, which takes hours on a chip this size.
- **A slow, disruptive load.** Each new bitstream is copied to the config SD over USB and the board rebooted. Everything stops while it reloads.
- **Cables and physical access.** To see a console or debug a processor you need serial adapters and a JTAG debug probe, wired to the right pins, and someone at the board. A design that hangs needs a hand on the power switch.

2.2 The answer: a harness and a partition

The FPGA in the MPS3 supports **partial reconfiguration** (AMD calls it Dynamic Function eXchange): one region of the chip can be rewritten while the rest keeps running. The platform uses it to split the FPGA in two.

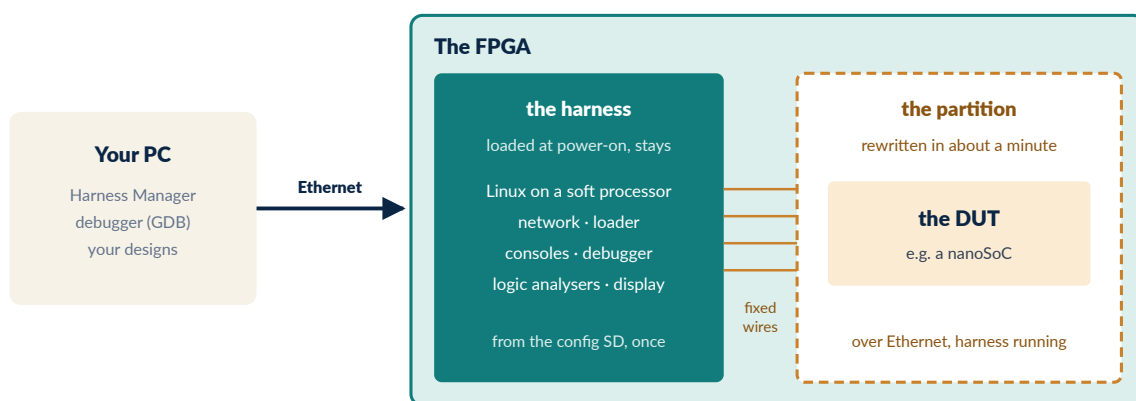


Figure 2.1. The harness and the partition. The harness is loaded once and keeps running; the partition holds the design under test and is rewritten from your PC. The two meet on a fixed set of wires: clock and reset, a console, a debug port, LEDs and switches, flash.

- **The harness** is loaded by the MCC at power-on, as in chapter 1, and then stays. It is a small computer made of FPGA logic: a soft processor running Linux, with the board's Ethernet, a loader that rewrites the partition, the consoles, a debugger and the display.
- **The partition** is the region left for your design. The design in it is called the **DUT**, the device under test. The harness rewrites the partition over Ethernet while it keeps running.

2.3 What it lets you do, and what it simplifies

WITHOUT THE HARNESS	WITH IT
Rebuild the whole FPGA for every change	Build only your design, for the partition: under an hour, and only when the logic changes. A program change needs no FPGA build at all (chapter 13).
Copy to the SD card, reboot the board	Load a design from your PC in about a minute; the network, consoles and debugger stay up.

WITHOUT THE HARNESS	WITH IT
Serial adapters, a JTAG probe, wiring	The consoles and the debugger come over the same Ethernet cable. You install only the debugger's front end on your PC.
A hung design needs a hand on the switch	Load the empty design back from your PC; the harness never stops answering.
Status lights and guesswork	The front panel shows what is loaded, whether it was verified and the board's name and address.

WHY

The fixed wires are the price of the speed. Every design for the partition must use the same boundary, and the harness and its designs must be built for each other. The platform checks that for you before every load (chapter 10).

3 nanoSoC

nanoSoC is SoC Labs' family of Arm Cortex-M systems-on-chip built on an AMBA AHB bus fabric, made for research, teaching and silicon. Members use different Cortex-M processors (Cortex-M0, Cortex-M0+ and Cortex-M4) around the same structure. The harness is built around it, so a nanoSoC drops straight into the partition.

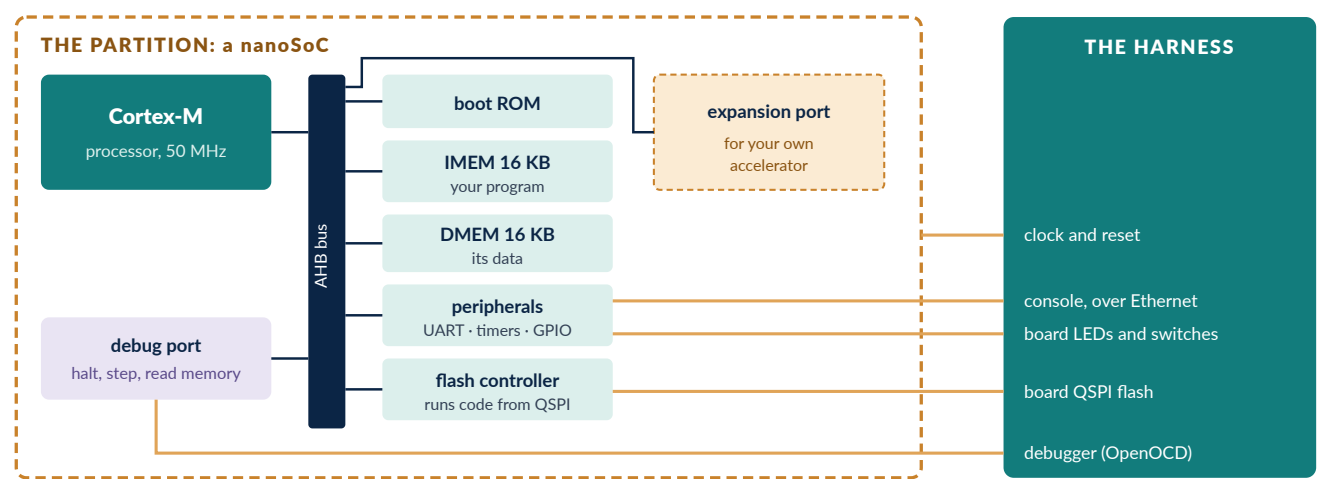


Figure 3.1. A single-core nanoSoC in the partition (simplified). The processor runs its program from instruction memory (IMEM). Its debug port, console UART, GPIO and flash pins leave the partition on the fixed wires, and the harness connects them to your PC and to the board.

3.1 What it is

Every nanoSoC is a complete microcontroller built on the same pattern: one or more Arm Cortex-M processors on an AHB bus fabric, a boot ROM, instruction and data memory, peripherals such as a UART, timers and GPIO, a controller that can run code straight from QSPI flash, and an Arm debug port. The `nanosoc` design this guide uses has one Cortex-M0 and 16 KB each of instruction and data memory. Its **expansion port** is an open slot on the bus, where you attach your own hardware, such as an accelerator. The members of the family share this core and differ in what they add: `nanosoc` is the single-core design this guide uses, and the User Guide covers the others, such as `nanosoc_upy`, which runs MicroPython from flash.

3.2 Why it exists

Designing a chip is hard to learn on a commercial SoC, which is too large to understand whole. nanoSoC is small enough to read end to end, uses real Arm IP under academic licence, and is set up so that the same design goes to simulation, to an FPGA and to silicon. A research group can add its own accelerator, check it on the MPS3 with real software, and then tape it out.

3.3 How the harness is built around it

The partition's fixed wires are exactly what a nanoSoC needs: a clock and reset, a JTAG debug port, a console UART, sixteen GPIO lines (wired to the board's LEDs and switches) and the QSPI flash. The harness turns each into something you can use from your PC: the debug port into a debugger session, the UART into a console over Ethernet. That is why a nanoSoC design loads like any other, with nothing to wire.

The nanoSoC designs contain Arm IP (Cortex-M processors, CoreSight and CMSDK), so the platform's download carries none of them. With Arm Academic Access you build **nanosoc** once yourself (chapter 11). Without it, chapter 10 still exercises the whole platform with open designs.

4 How you'll work

You drive the board from your PC with Harness Manager's window. This chapter tours it, and introduces the last three words you need before you start.

4.1 The Harness Manager window

Harness Manager is SoC Labs' program for the board. You open its window with **harness-manager app** (on Linux, also **Harness Manager** in the application menu). Its parts, from left to right and top to bottom (Figure 4.1):

- **The board list**, on the left. **+** adds a board by its address. Select a board and click **Open board**: the window then holds the board for you until you click **Close board**, top right.
- **The header** shows what the board runs: the harness's fingerprint (**Shell**), the design in the partition, the harness version and whether the Debug USB is connected.
- **Overview**: the board's health at a glance, and a copy of its front panel. **Live** shows the board's own picture, a few times a second, to look at only; **Text** gives its rows as text you can select and copy.
- **Workbench**: where you spend most of your time. You load designs at the top, read the design's console in the middle, and start the debugger on the right.
- **Build**: a guided build of your own FPGA design, in five steps (chapter 14).
- **Board**: the rarer jobs, one page each. **Recover** (from resetting the design to rebooting the board), **Versions**, **Connections**, **Readings**, **Access** (your claim and the board's name, below) and **About**.

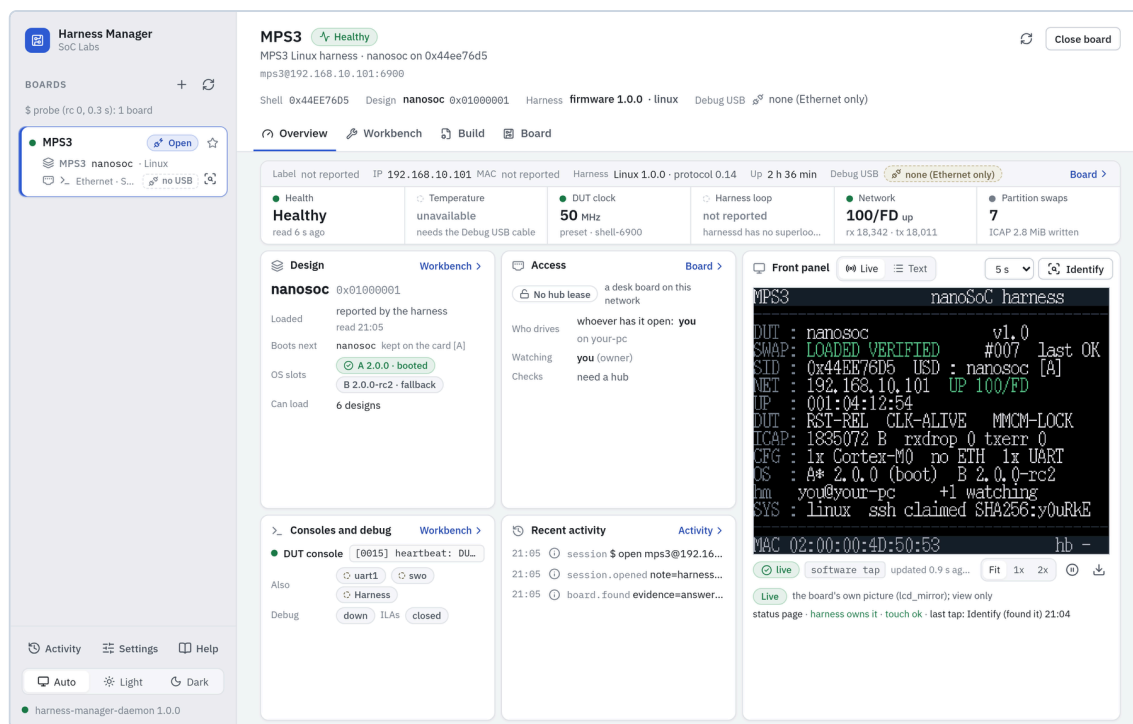


Figure 4.1. The window with a board open, on its Overview tab. Left, the board list; top, what the board runs; then the tabs. Taken from Harness Manager's demo (`harness-manager app --demo`, which needs no board), so some names and numbers differ from yours.

A step that stops what the design is doing asks for two clicks: tick **Arm**, then click the button. That keeps a stray click from resetting your work.

4.2 The command line: the same, for scripts

Every button has a command: `harness-manager <command>`. The window and the commands share one small background service on your PC, so you can mix them freely, for example by loading a design in the window and checking it from a terminal. This guide gives each step in the window first, then the same step as commands in a box headed *From the command line*. Use the commands to script tests, or on a PC with no screen. A few one-time set-up steps have no button yet; the guide says so where they come.

4.3 Three words

- design, overlay** A **design** is what you put in the partition, such as `nanosoc` or `led`. On disk it is an **overlay**: a directory with three files. The *partial* is the design itself; the *clearing* file empties the partition before the next design goes in (this FPGA family needs it); and `manifest.json` names the design and the harness it was built for. **Loading** a design (**Program** in the window) writes it into the partition.
- greybox** The empty design: the partition with nothing in it. The board starts with it, and it is the safe state to return to; Harness Manager calls it the *baseline*, and **Restore baseline** loads it.
- claim** Your board trusts one SSH key, the first one sent to it. Sending yours is **claiming** the board. From then on, the debugger and every change to the board's Linux work only with your key, so nobody else on your network can take them over.

5 What you need

Check your board first, then gather the rest. Ten minutes.

5.1 Your board's revision

The platform supports the MPS3 **revision C**. A revision B board boots the same release, but it has not been tested; revision A is not supported. Check yours in either of two ways:

- **On the board:** the printing near the FPGA's fan reads **HBI-0309C** (revision C) or **HBI-0309B** (revision B).
- **In the board's log:** connect the Debug USB to your PC and switch the board on. A drive named **V2M-MPS3** appears: the config SD. Open **LOG.TXT** on it; it records the board as, for example, **MotherBoard Revision C Variant A**. "Variant A" is not revision A.

5.2 Your board controller's firmware: check it, don't change it

The same **LOG.TXT** records the version of the MCC's firmware. Note it. The platform is proven on version 1.3.2; other versions are allowed, but untested. Do not update or downgrade the MCC to match.

WHY

The harness asks nothing of the MCC beyond loading the FPGA at power-on, so changing the MCC's firmware is a risk with nothing to gain. Harness Manager never writes the MCC's firmware.

5.3 The rest

ITEM	WHAT FOR
The MPS3's power supply and Debug USB cable	The Debug USB is needed for the first install only (chapter 7)
An Ethernet cable	Everything, every day: one cable from the board to your PC
A PC with a free Ethernet port	Linux, macOS or Windows, with git and Python 3.10 or newer
A microSD card, 4 to 32 GB, and a card reader	The harness's Linux and your settings live on it (chapter 8). Cards of 2 GB or less do not work; larger than 32 GB is untested
The platform release	The harness, its Linux and the open designs, from https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases (chapter 7)
Arm Academic Access and the nanoSoC source (optional)	Chapters 11 to 13. nanoSoC contains Arm IP, so you build the design yourself (chapter 11). The source comes from SoC Labs, on proof of your Arm Academic Access (contact SoC Labs support)
AMD Vivado 2026.1 and a licence for the board's FPGA (optional)	Chapters 11 and 14. The free tier does not cover this FPGA: you need the Core tier or higher. The build kit comes with the release

Harness Manager and the Arm compiler and debugger are free downloads; chapter 6 installs them.

6 Install your tools

Three things on your PC: an SSH key, Harness Manager, and Arm's compiler and debugger. Then connect the PC to the board. About 15 minutes, most of it downloads.

6.1 An SSH key

Your board will trust one SSH key: yours (chapter 9). If `~/.ssh/id_ed25519.pub` already exists, use it and skip this step. Otherwise make one, and press Enter to accept the file name:

```
ssh-keygen -t ed25519
```

Keep a copy of the key somewhere safe: a claimed board trusts only this key.

6.2 Harness Manager

You need git and Python 3.10 or newer. A Chrome-family browser (Chrome, Chromium, Edge or Brave) gives the window its own frame; without one it opens in a browser tab.

SYSTEM	INSTALL FIRST
Ubuntu 22.04 or 24.04, Debian 12	<code>sudo apt install python3-venv git</code>
Rocky, RHEL or Alma 8	<code>sudo dnf install python3.12 expat git tar</code> (the system Python is too old)
Rocky, RHEL or Alma 9	<code>sudo dnf install python3.12 git-core tar</code>
Fedora	<code>sudo dnf install python3 git-core tar</code>
macOS	git and Python 3.10 or newer, for example from Homebrew

- 1 **Download and install it**, with the serial support the Debug USB needs. It needs no root and takes about 20 seconds.

```
git clone https://github.com/SoC-Labs/HarnessManager HarnessManager
HarnessManager/scripts/install.sh --with-serial
```

```
...
Harness Manager 1.0.0 is installed.
```

On Windows, in PowerShell: `powershell -ExecutionPolicy Bypass -File HarnessManager\scripts\install.ps1 -WithSerial`.

- 2 **On Linux, join the group that owns serial ports**, then log out and back in. Without it, the Debug USB's ports refuse you.

```
sudo usermod -aG dialout $USER
```

- 3 **Put `~/local/bin` on your PATH**, in both files, because a login shell reads `~/.bash_profile` and an ordinary terminal reads `~/.bashrc`:

```
echo 'export PATH="$HOME/local/bin:$PATH"' >> ~/.bash_profile
echo 'export PATH="$HOME/local/bin:$PATH"' >> ~/.bashrc
```

- 4 **Check it** in a new terminal:

```
harness-manager version
```

```
1.0.1
```

To update Harness Manager later, run `git -C HarnessManager pull` and the same install command again.

- 5 **Open the window** to see that it starts: `harness-manager` app. Its board list stays empty until chapter 8, when the board first answers on the network. Close it for now.

6.3 The Arm compiler and debugger

Download the **Arm GNU Toolchain** for the `arm-none-eabi` target from developer.arm.com/downloads/-/arm-gnu-toolchain-downloads, unpack it, and put its `bin` directory on your `PATH` as in 6.2. It gives you `arm-none-eabi-gcc`, the C compiler for chapter 13, and `arm-none-eabi-gdb`, the debugger. Check both in a new terminal:

```
arm-none-eabi-gcc --version
arm-none-eabi-gdb --version | head -1
```

```
...
GNU gdb (Arm GNU Toolchain 13.3.Rel1 (Build arm-13.24)) 14.2.90.20240526-git
```

You install no OpenOCD, the program that drives the processor's debug port: it runs on the board, inside the harness.

6.4 Connect your PC to the board

The board answers at `192.168.10.101` until you give it an address of its own. Your PC needs a fixed address on the same network, `192.168.10.0/24`. There is no DHCP server on the cable, so a port left to set itself gets only a `169.254.x.x` address and never reaches the board. Windows and Linux PCs are supported; Linux is the most tested.

- 1 **Join a free Ethernet port on the PC to the board's Ethernet socket** with the cable. A USB Ethernet adapter works.
- 2 **Give that port the fixed address `192.168.10.1`**, netmask `255.255.255.0`, and no gateway. Any `192.168.10.x` works except `.101`, which is the board's. Leave the gateway empty: the cable leads only to the board, and a gateway there would send your other traffic to it.

Windows	Settings → Network & internet → Ethernet (the adapter cabled to the board) → IP assignment → Edit: Manual , IPv4 on, IP address <code>192.168.10.1</code> , Subnet mask <code>255.255.255.0</code> , Gateway empty, Save .
macOS	System Settings → Network → the Ethernet port → Details → TCP/IP : Configure IPv4 Manually , IP address <code>192.168.10.1</code> , Subnet mask <code>255.255.255.0</code> , Router empty.
Linux	Your desktop's network settings (GNOME: Settings → Network → the wired port's gear → IPv4): Manual , Address <code>192.168.10.1</code> , Netmask <code>255.255.255.0</code> , Gateway empty.

If another network on the PC also uses `192.168.10.x`, disconnect it.

- 3 **Let the board's replies in.** Every TCP connection (control, SSH, the consoles) is opened by your PC, so it needs no rule. Some of the board's UDP replies do: identify, which finds the board; the claim (chapter 9); and the board's rescue mode. A firewall that drops them makes those steps time out while everything else works.
 - **Windows: make the board's network Private.** **Settings** → **Network & internet** → **Ethernet** (the board's adapter) → **Network profile type** → **Private network**. *Why:* Windows treats a new network as Public, and on a Public network Windows Firewall drops the board's UDP replies.
 - **Windows: allow UDP from the board's network.** Even on a Private network the firewall can drop some of them, so add one inbound rule: **Windows Security** → **Firewall & network protection** → **Advanced settings** → **Inbound Rules** → **New Rule...**: **Custom**, **All programs**, protocol **UDP**, remote

IP address `192.168.10.0/24` , **Allow the connection**, name `Harness Manager board UDP` . It lets in UDP from the board's network and nothing else.

- **Windows: if a Windows Security alert asks about Python**, click **Allow**. **Cancel** makes a block rule for Python that wins over the rule above; to undo it, find the Python rule with the block sign in **Inbound Rules** and choose **Disable Rule**.
- **Linux with firewalld** (Fedora, Rocky, RHEL, Alma): trust the board's network, `sudo firewall-cmd --permanent --zone=trusted --add-source=192.168.10.0/24`, then `sudo firewall-cmd --reload`. Ubuntu's firewall is off unless you turned it on.
- **macOS**: if the firewall is on and asks whether Python may accept incoming connections, click **Allow**.

FROM THE COMMAND LINE: WINDOWS

In PowerShell as Administrator (Start, type `PowerShell` , right-click **Windows PowerShell**, **Run as administrator**). `Get-NetAdapter` names the adapter; below it is `Ethernet 2` .

```
Set-NetIPInterface -InterfaceAlias "Ethernet 2" -Dhcp Disabled
New-NetIPAddress -InterfaceAlias "Ethernet 2" -IPAddress 192.168.10.1 -PrefixLength 24
Set-NetConnectionProfile -InterfaceAlias "Ethernet 2" -NetworkCategory Private
New-NetFirewallRule -DisplayName "Harness Manager board UDP" -Direction Inbound `
    -Protocol UDP -RemoteAddress 192.168.10.0/24 -Action Allow
```

On Linux, `sudo ip addr add 192.168.10.1/24 dev <port>` sets the address until the PC restarts; `ip link` names the port (for example `enp3s0`).

4

For the command line, name the board in your terminal. The commands in this guide use `$B` . Set it in each new terminal:

```
B=192.168.10.101
```

In PowerShell, write `$B = "192.168.10.101"` , and set the guide's other names the same way (`$PORT = "COM7"`). Type a command that the guide splits with `\` on one line.

5

Check it. Your PC's side now: `ipconfig` (Windows) or `ip addr` (Linux) shows `192.168.10.1` on that port. The board's side, once the harness runs (chapter 7, step 6): `harness-manager info $B` answers with the board's shell. If it says `offline` , check the cable and the address; if it answers but a claim or a search for boards times out, check step 3. Chapter 15 has more.

7 Put the harness on the board

Once per board: through the Debug USB, you back up the config SD, copy the harness onto it, and reboot the board so that the MCC loads it. About 15 minutes.

COMMAND LINE ONLY

This one-time step has no button yet: the window has no one-step install of the harness onto a new board. Use the commands below, in a terminal. It is the only chapter done wholly on the command line.

First, download the release. From <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases>, download the harness bundle for v2.0 and unpack it into `~/mps3-bundle` . It holds:

```
~/mps3-bundle/config-sd/      the files for the config SD (this chapter)
~/mps3-bundle/linux_slot.img  the harness's Linux, with linux_bundle.json (chapter 8)
~/mps3-bundle/stage0_mkcard.py the tool that writes the microSD, with stage0_pack.py (chapter 8)
~/mps3-bundle/overlays/      the eight open designs, one directory each (chapter 10)
```

- 1 Connect the board.** With the board switched off and its user microSD slot empty, plug the Debug USB and the Ethernet cable into your PC, then switch the board on. The config SD appears on the PC as a drive named `V2M-MPS3`.
- 2 Find the MCC's serial port and the drive.** The Debug USB adds four serial ports; `probe` says which one is the MCC's console.

```
harness-manager probe
```

```
...
mps3@usb:serial:///dev/ttyUSB0    MPS3 on Debug USB (MCC console)    FT4232H 0403:6011 ...; V2M-
MPS3 volume /dev/sdb1 ...
```

Set both names for the commands below. If the drive is not mounted, open it once in your file manager.

```
PORT=/dev/ttyUSB0
DRIVE=/media/$USER/V2M-MPS3
```

	SERIAL PORT (THE MCC)	CONFIG SD DRIVE
Linux	<code>/dev/ttyUSB0</code>	<code>/media/\$USER/V2M-MPS3</code> (or <code>/run/media/\$USER/...</code>)
macOS	<code>/dev/cu.usbserial-...</code>	<code>/Volumes/V2M-MPS3</code>
Windows	a COM port, such as <code>COM7</code>	a drive letter, such as <code>E:\</code>

In these commands the target `-` means "this board, over the Debug USB only".

- 3 Back up the config SD.** This zip is your way back to the board exactly as it arrived. Keep it off the board.

```
harness-manager sd - --volume $DRIVE backup ~/mps3-backups
```

```
...
backed up 10 files from V2M-MPS3 to ~/mps3-backups/V2M-MPS3-20261002T125025Z.zip
sha256    a2fa5a92...
```

- 4 Install the harness.** It asks first, refuses without a backup, and never touches the MCC's own firmware. Answer `y`.

```
harness-manager sd - --volume $DRIVE install ~/mps3-bundle/config-sd \
--backup ~/mps3-backups/<the zip from step 3>
```

```
write 4 files to the SD of mps3@usb:... (backup ...) [y/N] y
...
written 4 files to the SD of mps3@usb:...
not running yet: reboot (`harness-manager mcc TARGET reboot`), then check `harness-manager
info TARGET`
```

A write over USB can take a few minutes. Slow is not failed: **do not unplug, switch off or start a second write.**

- 5 Reboot the board** so that the MCC loads the harness. It asks first, then waits until the board has gone down and come back. On Linux, eject the drive first.

```
harness-manager mcc - --serial $PORT reboot
```

```
reboot mps3@usb:serial://...? The board reloads from its SD and the running design is lost
[y/N] y
...
rebooted mps3@usb:serial://... (seen: sent, down, up)
MCC loaded MB/HBI0309C/Nanosoc/nanosoc.bit
```

On a revision B board the last line names `HBI0309B`. The file is called `nanosoc.bit` because that is where the MCC looks for it; it is the harness, with an empty partition.

- 6 Check it over Ethernet.** Wait a minute, then:

```
harness-manager info $B
```

```
board      MPS3 at 192.168.10.101:6900
shell      0x44ee76d5 harness ? (?)
...
control    rescue
note       stage0 says: card has no stage0 slots
...
```

`rescue` is right for now. The harness is running, but its Linux lives on the user microSD, which is still empty, so its first stage (*stage-0*) answers and waits. Chapter 8 gives it Linux. You can unplug the Debug USB: from now on you need only Ethernet.

If `mcc` says `no permission to open serial port`, your user is not in the serial ports' group yet: see 6.2, step 2.

8 Put Linux on the microSD

The harness boots its Linux from the user microSD. You write the whole card on your PC, once, put it in the board, and switch on. About 15 minutes.

The card holds Linux twice, in two *OS slots*, A and B, so that a later update can be tried in one slot while the other stays safe. It also holds an empty partition called `/persist`, where the board keeps your claim and its name (chapter 9).

No card reader? User Guide chapter 2 puts Linux on the card over Ethernet instead, in about an hour and a quarter.

Writing the card has no button yet, so steps 1 and 2 are commands. From step 3 on, you work in the window.

- 1 Make the card image.** On Windows type `py` for `python3`; on Rocky, RHEL or Alma 8 type `python3.12`.

```
cd ~/mps3-bundle
python3 stage0_mkcard.py card --slot-a linux_slot.img --out-dir card \
  --card-img card.img --card-mib 512
python3 stage0_mkcard.py check card.img
```

```
...
-> stage0 would boot slot A
```

`check` reads the image the way the board's first stage will. Ignore the `dd` lines the first command prints: they write the parts one by one, and step 2 writes the whole image at once.

- 2 **Write the image to the microSD** in your card reader. About a minute. **This erases the microSD**, and `dd` erases whatever disk you name: check the size twice, and never name your PC's own disk or the config SD (V2M-MPS3). Below, `sdb` and `disk4` stand for the names your PC gives the card.

	FIND THE MICROSD, AND UNMOUNT IT	WRITE THE IMAGE
Linux	<code>lsblk</code> lists it by size, for example <code>sdb</code> ; then <code>sudo umount /dev/sdb?*</code>	<code>sudo dd if=card.img of=/dev/sdb bs=4M conv=fsync status=progress</code>
macOS	<code>diskutil list</code> shows it as external, for example <code>disk4</code> ; then <code>diskutil unmountDisk /dev/disk4</code>	<code>sudo dd if=card.img of=/dev/rdisk4 bs=4m</code> , then <code>diskutil eject /dev/disk4</code>
Windows	A raw image writer, one that copies an <code>.img</code> file onto a whole card: choose <code>card.img</code> and the microSD's drive, then write.	

- 3 **Boot from the microSD.** Switch the board off, put the card in the **user microSD** slot, and switch it on. Linux takes 3 to 4 minutes to start.
- 4 **Add the board to the window, once.** Open the window (`harness-manager` app), click **+** above the board list, choose **By address**, type `192.168.10.101` and click **Add** (Figure 8.1). Tick **Save to boards.toml** first if you want the window to list the board every time it starts.

Figure 8.1. Adding the board by its address.

- 5 **Open it.** Select the board in the list and click **Open board**. The header now shows the shell `0x44EE76D5`, the design `greybox` and the harness `firmware 1.0.0 · linux`, with the label **Healthy**: Linux is running and ready. If the board does not answer after 5 minutes, see chapter 15.

FROM THE COMMAND LINE

```
harness-manager info $B
```

```
board      MPS3 at 192.168.10.101:6900
shell      0x44ee76d5  harness 1.0.0 (...)
design      greybox (0x00000000)
```

```
...
```

```
control    idle
```

```
...
```

`control idle` means Linux is running and ready. `rescue` means the harness found no Linux on the card.

WHY THE WAIT

The harness's processor is built from FPGA logic and runs at 100 MHz, so its Linux takes a few minutes to start. That happens only at power-on; loading designs later does not restart it.

9 Claim and name your board

Claim the board with your SSH key, so that only you can debug it and change its Linux, then give it a name of its own. About 15 minutes.

9.1 Claim it

A new card takes two claims. The board keeps a claim in `/persist`, which is still empty, and the harness never formats it by itself. So the first claim lives only in memory: it lets you log in and format `/persist`. The second, after a restart, is kept.

CAUTION

The first key claims the board for good. Once the claim is kept on the card, nothing on the network can undo it or add another key; only a command typed at the harness's serial console releases it (User Guide chapter 2). Claim with the key you keep, both times.

1

Claim it, so that your key can log in. In the window, open **Board → Access**. The **SSH claim** card says **Not claimed**. Click **Claim with my key...**, read what it says, then click **Claim with my key**. It uses `~/.ssh/id_ed25519.pub`. The card then says **Claimed by you** (Figure 9.1).

FROM THE COMMAND LINE

```
harness-manager board claim $B --key ~/.ssh/id_ed25519.pub
```

```
claim mps3@192.168.10.101:6900 with key SHA256:... (../id_ed25519.pub)? That key gets root on
the board, the board refuses every later claim, and its slot changes then need that key
(undo: mps3-unclaim on the serial console) [y/N] y
```

```
...
```

```
claim      mps3@192.168.10.101:6900: claimed by you (SHA256:...)
host key    SHA256:... pinned (boards.toml ...)
```

Harness Manager also records the board's *host key*, the key the board proves who it is with, and from now on refuses a board at this address that answers with another one.

10 Your first designs

Two open designs prove the platform end to end: `led` counts on the board's LEDs, and `uart_echo` answers on a console. About 10 minutes.

Once: show Harness Manager your designs. Copy the eight open designs from the download into one directory of your own; the designs you build later go there too:

```
mkdir -p ~/mps3-overlays
cp -r ~/mps3-bundle/overlays/* ~/mps3-overlays/
```

Then, in the window, open **Settings** (bottom left) → **Harness & kits**, and add the directory's full path (for example `/home/you/mps3-overlays`) under **Extra overlay directories, searched first**. A single design can also be added from the Workbench with **Import...**

FROM THE COMMAND LINE

```
harness-manager config set mps3.overlay_dirs ~/mps3-overlays
```

- 1 **See what the board runs.** The window's header shows it. **Shell 0x44EE76D5** is the harness's fingerprint, its *static ID*: every design is built for one harness, and Harness Manager compares the two before every load. **Design greybox**: the partition is empty.

FROM THE COMMAND LINE

```
harness-manager info $B

board      MPS3 at 192.168.10.101:6900
shell      0x44ee76d5  harness 1.0.0 (...)
design      greybox (0x00000000)
build      firmware/fabric check: ok
...
control    idle
ssh claim   claimed by you (SHA256:...)
...
```

- 2 **Load `led`.** Open the **Workbench**. In **Design to program**, pick `led`. Harness Manager runs a *preflight* at once, which checks that the design fits this harness, and shows the result under the picker: **6 checks OK**, then one chip per check. The picker also tags some designs, for example **loaded now** and **baseline** beside `greybox`. Tick **Arm**, then click **Program** (Figure 10.1). About 35 seconds.

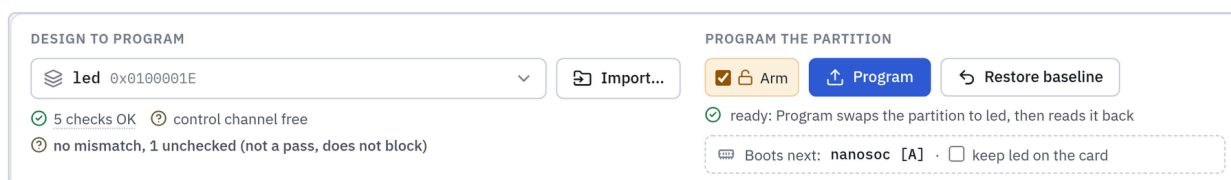


Figure 10.1. Ready to load `led`: the preflight passed, **Arm** is ticked. From the demo.

A green box then reports the result:

```
Programmed led 0x0100001E · 35 s in all · pushed 1.3 MiB in 31 s · 43 KB/s · verified ...
```

verified means the harness read the design's own ID back from the running design. Four of the board's LEDs now count in binary. The preflight's last chip, **no mismatch, 1 unchecked (not a pass, does not block)**, is expected: only an external JTAG cable could read that value.

FROM THE COMMAND LINE

```
harness-manager program $B led
```

```
preflight:
  ok      control channel free 192.168.10.101:6900 answered (it serves one client at a
time)
  ok      shell_id matches 0x44ee76d5
  ...
  unchecked static_usercode matches needs JTAG: ... (not a pass)
program led (0x0100001e) into mps3@192.168.10.101:6900? [y/N] y
  ...
programmed led (0x0100001e) into mps3@192.168.10.101:6900 in 36.6s via tcp; verified
```

3

Load `uart_echo` and use its console. Load it as in step 2 (about 37 seconds). The **DUT console** under the program strip is the design's console; it says **up • uart0 • over Ethernet**. Click in the console and type **hello**: each letter comes back from the design. The design runs as soon as it is loaded, so no reset is needed. To see its greeting as well (optional), tick **Arm** in the console's toolbar and click **Reset DUT**: the console shows **rm_uart_echo ready**.

FROM THE COMMAND LINE

```
harness-manager program $B uart_echo --yes
harness-manager console $B uart0
```

```
console uart0 of mps3@192.168.10.101:6900: typing goes to the board, Ctrl-] exits (Ctrl-C is
sent to the board); Ctrl-] then r, r resets the DUT
```

Optionally, reset the DUT from the console to see the greeting: **Ctrl-]**, then **r**, then **r** again to confirm.

```
reset the DUT of mps3@192.168.10.101:6900: done
rm_uart_echo ready
```

Type **hello**, then press **Ctrl-]** to leave. Ctrl-C does not leave: it is sent to the board, as a real serial terminal would send it. **--yes** answers **program**'s question for you.

WHY A RESET SHOWS THE GREETING

The harness passes a design's console output on only while a console is open, and can hold only 16 bytes until then. A greeting printed before you connect is lost; the design itself is running, and answers what you type. The window's console is open whenever the board is, so its **Reset DUT** always catches the greeting. On the command line the rule is: **open the console first, then reset the DUT**.

You have now loaded designs and used a console, all over Ethernet. With Arm Academic Access, go on: chapter 11 builds the nanoSoC design. Without it, skip to chapter 14, which builds a design of your own from an open one.

11 Build the nanoSoC design

nanoSoC contains Arm IP, so the platform's download cannot include it. If you have Arm Academic Access, you build the `nanosoc` design once, from its source, with Harness Manager's kit. About an hour, most of it Vivado working on its own.

NO ARM ACADEMIC ACCESS?

Skip chapters 11 to 13: they all use `nanosoc`. Keep using the open designs, and go on to chapter 14, which builds a design of your own from an open one.

11.1 What you need

- **The nanoSoC source.** It comes from SoC Labs, on proof of your Arm Academic Access (contact SoC Labs support). With it come SoC Labs' QSPI controller (`ahb_qspi`), the platform files that fit nanoSoC into the partition (`platform`), and the platform's *design file* for nanoSoC, `nanosoc.json`. The design file tells Harness Manager every source file (246, in the order Vivado must read them), the program that fills the instruction memory, and the design's ID.
- **The Arm IP** that nanoSoC uses (the Cortex-M0, CoreSight, CMSDK and a few peripherals), from your Arm Academic Access. The build reads it where it is; nothing is copied.
- **The Arm GNU toolchain and CMake**, to build nanoSoC's boot ROM before Vivado reads it.
- **AMD Vivado 2026.1 exactly, with a licence for the board's FPGA** (the KU115: the Core tier or higher), and **the build kit** for this harness (`0x44EE76D5`), attached to the v2.0 release on <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases>. Chapter 14 says more about both.
- A machine with 8 GB of memory: the build peaks at about 5 GB.

11.2 Lay out the folder

Every path in `nanosoc.json` is relative to the folder the file sits in, so you never edit it. You only put four things next to it, three from the nanoSoC source and one link:

```
~/nanosoc/
nanosoc.json
nanosoc_m0_soc/    the nanoSoC source
ahb_qspi/          the QSPI controller
platform/          the wrapper and the hello program
arm_ip_library →   your Arm IP (a link)
```

- 1 **Make the folder.** Put `nanosoc.json`, `nanosoc_m0_soc`, `ahb_qspi` and `platform` from the nanoSoC source in `~/nanosoc`.
- 2 **Link your Arm IP.** `ARM_IP_LIBRARY_PATH` is wherever your Arm IP lives. The link must hold these five folders, as Arm ships them: `latest/Cortex-M0`, `latest/Corstone-101`, `SoC-400/coresight_soc`, `PL022/PL022-BU-00000-r1p4-00rel0` and `CG092/CG092-BU-50000-r2p0-00rel0`. If yours are elsewhere, make `arm_ip_library` a folder of links with these names; a missing file stops the build at its first check, which names it.

```
cd ~/nanosoc
ln -s "$ARM_IP_LIBRARY_PATH" arm_ip_library
```

- 3 **Build the boot ROM.** nanoSoC's first instructions live in a small ROM that is generated from C, so build it before Vivado reads it:

```
make -C nanosoc_m0_soc/pynq firmware
```

WHY A LINK, NOT A COPY

Your Arm IP licence lets you use the IP, not redistribute it, so the design file names it only through a link that you make. The same link means one design file works on every machine.

11.3 Build it

This build uses the command line: the Build tab's **Paste .json** has no folder to start a relative path from. The window shows the result in the last step.

- 1 **Import the kit** (once per harness version):

```
harness-manager kit import ~/Downloads/<the kit zip>
```

- 2 **Write the build directory.** It warns that the ID `0x01000001` is in the platform's own range: expected, because this *is* the platform's design. Keep the name and ID: the board recognises nanosoc by them, and picks its debugger set-up by the ID (chapter 12).

```
cd ~/nanosoc
harness-manager kit script --static-id 0x44EE76D5 --design nanosoc.json --out
~/builds/nanosoc --jobs 4
```

- 3 **Build.** `kit build` prints the Vivado command; run it exactly. About 50 minutes.

```
harness-manager kit build ~/builds/nanosoc
```

- 4 **Check and add.** The verdict is **passed**, with a positive WNS: every timed path meets the 50 MHz clock. Vivado prints about 18 CRITICAL WARNINGS about the partition's constraints during the build: they are expected. The boundary check reports `the static->RM boundary was timed`: that is the pass you want. `kit pack --import` adds the design to your store, and `nanosoc` appears in the Workbench beside the open designs.

```
harness-manager kit check ~/builds/nanosoc
harness-manager kit pack ~/builds/nanosoc --import
```

```
HM_RM_BUILD_COMPLETE rm=nanosoc rm_id=0x01000001 static_id=0x44EE76D5 ...
the build nanosoc: passed (unchecked is not a pass: see the list)
...
imported into the store (...): it shows in Program
```

You never need to build it again for this harness; chapter 12 loads it.

CAUTION: NOT THE BUILT-IN EXAMPLE

Harness Manager also has a built-in design called `nanosoc` (`--design nanosoc`, or the Build tab's example). It describes only the partition's wires, with no nanoSoC inside. Harness Manager 1.0.0 refuses to build it; always build from `nanosoc.json`.

SoC Labs has built `nanosoc` from this design file with Harness Manager 1.0.0: every check passed, with 3.3 ns to spare at 50 MHz, in about 50 minutes. The design file names the hello program as the instruction memory's contents; to build a different program in, see *User Guide* §6.3.

12 Meet nanoSoC on the board

Load nanoSoC, read the greeting its program prints, then halt its Cortex-M0 in the debugger and look at its registers. About 10 minutes. You need the `nanosoc` design you built in chapter 11 and the claim from chapter 9.

1 Load nanoSoC. In the **Workbench**, pick `nanosoc`, tick **Arm** and click **Program**. About a minute; the green box then reads **Programmed nanosoc 0x01000001 • 63 s in all • ... • verified**. The design comes with a small program already in its instruction memory: *hello*, which prints a greeting once when it starts, then waits in a loop.

2 Reset the DUT and read the greeting. In the console's toolbar, tick **Arm** and click **Reset DUT**. The **DUT console** shows:

```
Hello world
** TEST PASSED **
```

That is the Cortex-M0 running.

3 Start a debug session. In the **Debug** card on the right, click **Open session**. Harness Manager starts OpenOCD, the debug server, on the board, and connects it to your PC through your claim. After about 50 seconds the card turns **up**, says OpenOCD runs **on the board**, and shows the local port GDB connects to, such as **gdb 127.0.0.1:43571** (Figure 12.1). **Ports: telnet on the board only • tcl on the board only** means OpenOCD's other two ports stay on the board, out of reach of the network; GDB is all you need. The console stays open beside it. While the session runs, the window's header can show **Last warning**, about a read **over the 2 s read time**: the board's OpenOCD kept the harness busy for a moment, and it is harmless.

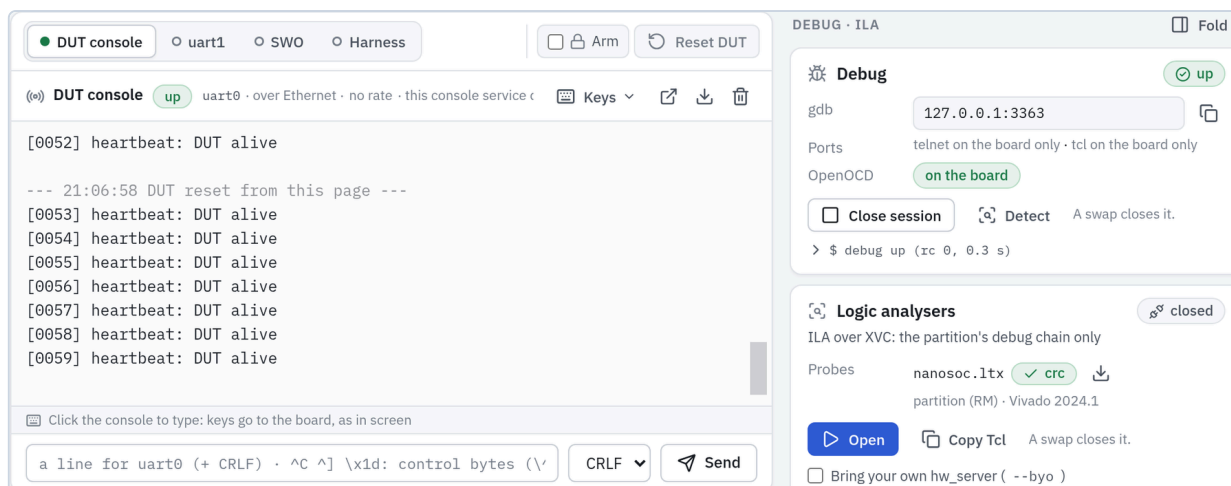


Figure 12.1. The DUT console after a reset, and the **Debug** card with a session up. The copy button beside the port copies the whole GDB command. From the demo: its console text and port differ from yours.

- 4 **Halt the processor and read two registers.** GDB is a terminal program, so this step is always on the command line. Click the copy button beside the port, and paste the command *on its own* at a terminal's prompt: it already starts with `arm-none-eabi-gdb`, so type nothing before it (`gdb` in front would start your PC's own GDB instead), and paste it once. If GDB stops with `--Type <RET> for more, q to quit, c to continue without paging--`, type `c`. Then type:

```
arm-none-eabi-gdb -ex "set remotetimeout 60" -ex "target extended-remote 127.0.0.1:43571"
(gdb) monitor halt
(gdb) info registers pc sp
```

pc	0x378	0x378
sp	0x1800fbf0	0x1800fbf0

```
(gdb) detach
(gdb) quit
```

`pc`, the program counter, is where the processor is: `0x378` is hello's final loop. `sp` is its stack pointer, in data memory at `0x1800...`. The first halt can take up to a minute, and warnings that `keep_alive()` was not invoked in the 1000 ms `timelimit` (by 1 to 5 seconds) are normal: every request travels to the board and back. If you see `pc 0xfffffffffe` instead, the processor has locked up, usually because the board's flash holds a MicroPython boot image (and there was no greeting in step 2): see chapter 15.

- 5 **End the session:** **Close session** in the Debug card. The **Overview** then says **Debug down • ILAs closed**.
- 6 **Put greybox back:** tick **Arm**, then click **Restore baseline**. About 40 seconds; the green box reads **Restored baseline: greybox 0x00000000 • 39 s in all • ... • verified**.

FROM THE COMMAND LINE

Load, then read the console. The console must be open before the reset (Ctrl-], r, r), and must be closed again (Ctrl-]) before `debug up`: on the command line, one command uses the board at a time.

```
harness-manager program $B nanosoc --yes
harness-manager console $B uart0
```

```
...
programmed nanosoc (0x01000001) into mps3@192.168.10.101:6900 in 65.0s via tcp; verified
...
reset the DUT of mps3@192.168.10.101:6900: done
Hello world
** TEST PASSED **
```

Start the debug session in one terminal; it stays in the foreground until you press Ctrl-C. Paste its `attach` line into a second terminal, as in step 4. Your port number will differ.

```
harness-manager debug up $B
```

```
debug      mps3@192.168.10.101:6900: up
where      the board: OpenOCD runs there; gdb reaches it through the board's SSH
gdb        127.0.0.1:43571
attach     arm-none-eabi-gdb -ex "set remotetimeout 60" -ex "target extended-remote
127.0.0.1:43571"
config     nanosoc_mps3_jtag.cfg nanosoc_ops.tcl
...
```

End it with Ctrl-C, then put greybox back:

```
harness-manager restore $B
```

```
...
restored   mps3@192.168.10.101:6900 to the baseline (0x00000000) in 41.6s; verified
```

You have loaded three designs, used the DUT console, and halted a Cortex-M0 in GDB, all over one Ethernet cable. Chapter 13 puts your own program on it.

13 Your own program

Take the example program from the platform's repository, change its message, build it on your PC, load it into the running nanoSoC with GDB and check in GDB that it runs. No FPGA build is involved, so one round takes a minute or two. About 15 minutes the first time.

NOTE

This chapter is written for platform v2.0.1 or later, whose debugger loads this program in a second or two. On v2.0.0 the same commands work, but the load is slower: about 50 seconds per kilobyte, so about 20 seconds for this program.

KNOWN LIMITATION: NO CONSOLE OUTPUT YET

In v2.0, a program loaded this way runs, but its messages do not yet reach the DUT console. So this chapter checks your program in GDB instead (§13.3, step 3).

13.1 Get the example

The example, `nanosoc_hello`, is in the platform's public repository. Copy it into a directory of your own:

```
git clone https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform ~/mps3-platform
cp -r ~/mps3-platform/examples/nanosoc_hello ~/myhello
cd ~/myhello
```

It is four short files, all written for this platform: `main.c`, the program; `startup.S`, the vector table and the reset handler, which prepares memory for C and calls `main()`; `nanosoc_hello.ld`, the memory map (code in the instruction memory, IMEM, 16 KB at `0x1000_0000`; data and the stack in the data memory, DMEM, 16 KB at `0x1800_0000`); and a `Makefile`. It needs no C library and no vendor code.

It is built for the `nanosoc` design's Cortex-M0 (`-mcpu=cortex-m0`); other nanoSoC members use other Cortex-M cores. You also need `make`: on Ubuntu or Debian, `sudo apt install make`; on Rocky, RHEL or Fedora, `sudo dnf install make`.

13.2 Change it and build it

Open `main.c` and change the message near the top:

```
/* ---- Change me ----- */
#define MESSAGE "My first nanoSoC program"
```

Then build it:

```
make
```

```
...
text      data      bss      dec      hex filename
428        0         4      432      1b0 nanosoc_hello.elf
```

The 428 bytes of `text` are the whole program: what the debugger will write into IMEM. Your compiler version may give a few bytes more or less.

WHY THE PROGRAM SETS THE CONSOLE UP ITSELF

A UART sends each bit for `BAUDDIV` clock cycles. The design runs at 50 MHz and the harness console listens at 76,800 baud, so `console_init()` sets $50,000,000 / 76,800 = 651$. The 2 ms pause after each character is there because the harness collects console output every few milliseconds and holds only 16 bytes in between.

13.3 Load it and run it

Load the `nanosoc` design you built in chapter 11 first (as in chapter 12) if it is not loaded. The window gives you the console and the debug session side by side; GDB, which loads and starts your program, runs in a terminal.

- 1 **In the window's Workbench:** click **Open session** in the **Debug** card. Copy the GDB command with the button beside the port.
- 2 **In a terminal, the debugger.** In `~/myhello`, paste the copied command at the prompt, once, with nothing typed before it, and add `nanosoc_hello.elf` after `arm-none-eabi-gdb`. Answer GDB's pager prompt, if it appears, with `c`. Then:

```
arm-none-eabi-gdb nanosoc_hello.elf -ex "set remotetimeout 60" -ex "target extended-remote
127.0.0.1:43571"
(gdb) monitor reset halt
(gdb) monitor nanosoc.cpu0 arp_examine
(gdb) monitor halt
(gdb) load
Loading section .text, size 0x1ac lma 0x10000000
Start address 0x10000040, load size 428
(gdb) monitor reg pc 0x10000040
(gdb) monitor reg sp 0x18004000
(gdb) monitor reg xPSR 0x01000000
(gdb) monitor resume
```

Warnings such as `keep_alive() was not invoked in the 1000 ms timelimit` are expected: the board's OpenOCD pauses for 1 to 5 seconds at a time. After `monitor reset halt` you may also see `JTAG-DP STICKY ERROR`; the next two lines deal with it.

- 3 **Check that it runs.** Wait a few seconds, then reconnect, which halts the processor wherever it is, and look:

```
(gdb) detach
(gdb) target extended-remote 127.0.0.1:43571
(gdb) print hello_count
(gdb) info registers pc sp
```

`hello_count` is the number of lines your program has sent, so it grows by about one a second, and `pc` is inside your program (`0x1000...`). Type `continue` to let it run on.

FROM THE COMMAND LINE

Use two terminals. Terminal 1, the debug session: copy its `attach` line. Terminal 2: GDB, as in steps 2 and 3.

```
harness-manager debug up $B
```

WHY THESE COMMANDS

`monitor reset halt` resets the processor and asks the debugger to stop it. If the board's flash holds a boot image the processor cannot start, the processor may lock up straight after the reset, before the debugger has caught it. `arp_examine` makes the debugger look at the processor again, and `monitor halt` then stops it inside its boot ROM, before it reaches the flash. The program is linked to run in IMEM, at `0x1000_0000`, so `load` writes it there. Its first two words are the vector table: the starting stack pointer (`0x18004000`) and the address of its first instruction (`0x10000041`; the final 1 marks Thumb code, so the address is `0x10000040`). The three `monitor reg` commands start the program as the processor itself would after a reset, and `monitor resume` lets it go.

13.4 Debug it, and go round again

GDB has read `nanosoc_hello.elf`, so it knows your source. When the processor is halted (after you reconnect, or press Ctrl-C), try `print hello_count`, `info registers pc sp`, `list main` or `backtrace`.

To change the program: edit `main.c`, run `make` in another terminal, then in GDB repeat step 2 from `monitor reset halt`. GDB stays attached the whole time. When you finish, type `detach` and `quit`, and click **Close session** (on the command line, press Ctrl-C in terminal 1).

Your program lives only in the processor's memory: loading any design, or restoring greybox, replaces it. To make a program part of a design, so that it runs at every load, see User Guide chapter 6.

14 Your first FPGA design

Optional

Change a little FPGA logic, build it for the partition with Harness Manager's kit and AMD Vivado, check it and load it. About an hour, most of it Vivado working on its own.

Chapter 13 changed software. Here you change hardware, with the same kit and Build tab that chapter 11 used for nanoSoC, but from an open design that anyone may build: the `led` design from chapter 10. Building for the partition is much quicker than building a whole FPGA, because the harness is already built: Vivado places only your design, into the space and onto the fixed wires the harness left for it.

14.1 What you need

- **AMD Vivado 2026.1**, exactly that release, with the Kintex UltraScale device files. A design built for the harness must use the release the harness was built with.
- **A Vivado licence that covers the board's FPGA** (the KU115): the Core tier or higher. The free tier does not cover it.
- **The build kit for your harness** (static ID `0x44EE76D5`): the built, locked harness and the partition's constraints, in one zip. It is attached to the v2.0 release on <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases>.
- A machine with 8 GB of memory, and the platform's source, from <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform>.

14.2 Change the design

- 1 **Copy the template** into a directory of its own, under a new name:

```
git clone https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform ~/mps3-platform #
skip if you cloned it in chapter 11 or 13
mkdir ~/my_led && cd ~/my_led
cp ~/mps3-platform/fpga/dfx/rms/rm_led/rm_led.sv my_led.sv
```

- 2 **Make the change.** In `my_led.sv`, rename the module, and make it count on all eight LEDs. The full count now takes about 5 seconds; the lowest LEDs change too fast to follow:

```
module my_led #(
    parameter int NGPIO      = 16,
    parameter int COUNTER_W  = 28,
    parameter int BLINK_BITS = 8
    ...
)
```

The design counts with a free-running counter clocked at 50 MHz and shows its top `BLINK_BITS` bits on the LEDs; a wider counter counts more slowly.

- 3 **Describe it to Harness Manager** in `~/my_led/my_led.json`: the boundary groups it uses (clock and reset, status, GPIO), its file and its top module.

```
{
  "kind": "rm", "name": "my_led",
  "use": {"clkrst": {}, "status": {}, "gpio": {}},
  "clocks": [],
  "wrapper": "my_led.sv",
  "build": {"top": "my_led", "sources": ["my_led.sv"]}
}
```

14.3 Build it in the Build tab

The window's **Build** tab walks you through the build in five steps (Figure 14.1). Harness Manager never runs Vivado itself: it writes the build, gives you the command, and follows the build's log while it runs.

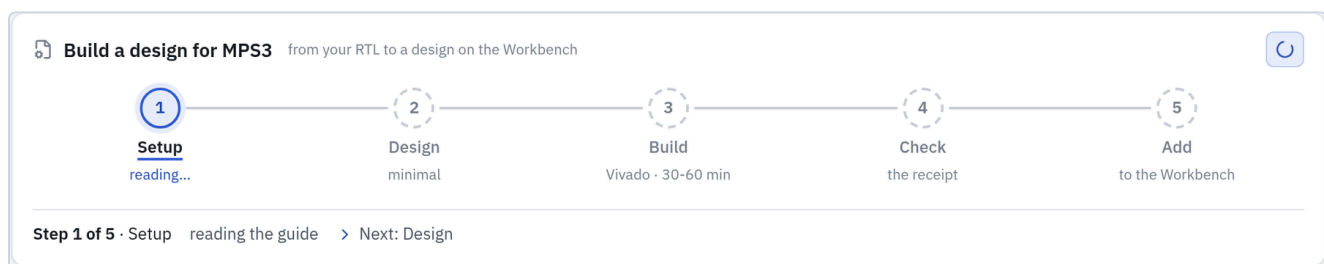


Figure 14.1. The Build tab's five steps, from the demo.

- 4 **Setup.** Under **Kit source**, choose **A folder or zip** and give the path of the kit zip you downloaded. The step also checks that Vivado 2026.1 starts and names any problem with it or its licence.
- 5 **Design.** Choose **Paste json** and paste `my_led.json`, with the source's full path (`"sources": ["/home/you/my_led/my_led.sv"]`). Harness Manager checks your design against the partition's boundary and proposes an `rm_id`, your design's own 32-bit ID, which the harness reads back after every load. Put the proposed value in both places: `"rm_id": "0x0100A88A"` in the JSON, and in `my_led.sv`, where the template sets `RM_ID_LED : 32'h0100_A88A`. Then click **Continue to Build**.
- 6 **Build.** Give a build directory, such as `/home/you/builds/my_led`, and click **Write the build directory**. Under **Run it your way**, keep **Batch**, copy the command, and run it in a terminal, exactly as given. Vivado works on its own for 30 to 60 minutes (this design took about 45 minutes on a busy server); the tab shows each stage as the log reports it, and moves on to Check when the build's receipt appears.
- 7 **Check.** Harness Manager reads the receipt without Vivado or the board: the build's own checks, the `rm_id`, the harness it was built for, and timing. **passed** is the verdict. A positive WNS (worst negative slack) means every timed path in your design meets the 50 MHz clock with time to spare. One check, `static_binding`, stays *unchecked* by design: a partial bitstream carries no record of its harness, and the build's own checks already proved the match.
- 8 **Add.** Click **Add to the Workbench and program...**. The Workbench opens with `my_led` picked: tick **Arm** and click **Program**. The preflight checks it as it did `led`'s, and the load ends **verified** when the harness reads back your `rm_id`. All eight LEDs now count. When you have seen it, **Restore baseline**.

WHY NOT VIVADO'S EXIT CODE

Vivado finishes with success even when one of the build's own checks has failed. Each check prints an `HM_GATE` line, and the build writes a receipt; the receipt and the build's last `HM_RM_BUILD_` line are the verdict. That is what the Check step reads. `harness-manager kit guide --why <gate>` explains how to fix a failed check.

FROM THE COMMAND LINE

The same five steps, in `~/my_led.kit guide` proposes the `rm_id`; `kit build` prints the Vivado command, which you run exactly.

```
harness-manager kit import ~/Downloads/<the kit zip>
harness-manager kit guide --static-id 0x44EE76D5 --design my_led.json
```

```
...
done 4 Wrapper and XDC my_led: 47 ports / 148 bits checked; rm_id 0x0100A88A (warning: no
rm_id: HM proposes 0x0100A88A)
...
```

Put the proposed ID in both files, as in step 5: `"rm_id": "0x0100A88A"` in `my_led.json`, and `RM_ID_LED = 32'h0100_A88A` in `my_led.sv`. Then:

```
harness-manager kit script --static-id 0x44EE76D5 --design my_led.json --out ~/builds/my_led
harness-manager kit build ~/builds/my_led
```

```
<vivado-install>/2026.1/Vivado/bin/vivado -mode batch -source ~/builds/my_led/build_rm.tcl -log
~/builds/my_led/build_rm.log -journal ~/builds/my_led/build_rm.jou
...
```

Run that command; then the verdict, the last log line that *starts* with `HM_RM_BUILD_`:

```
grep -E '^HM_RM_BUILD_' ~/builds/my_led/build_rm.log | tail -1
```

```
HM_RM_BUILD_COMPLETE rm=my_led rm_id=0x0100A88A static_id=0x44EE76D5
partial=~/out/my_led_partial.bin
```

```
harness-manager kit check ~/builds/my_led
```

```
the build my_led: passed (unchecked is not a pass: see the list)
ok      build      the build passed 24 gates
ok      rm_id      the netlist drives 0x0100A88A
ok      static_id   built against static 0x44EE76D5 (...)
ok      timing      your RM's paths: setup WNS 16.033 ns, hold WHS 0.077 ns
...
unchecked static_binding      a partial carries no static identity: ...
```

Read the `boundary_timing` line: `ok ... the static<->RM boundary was timed` means the build checked your design's own paths and the wires where it meets the harness. A `WARNING: boundary not timed` means the build directory came from an older script: write it again with `kit script ... --out ~/builds/my_led` and rebuild.

```
harness-manager kit pack ~/builds/my_led --import
harness-manager program $B my_led
```

```
overlay my_led (0x0100A88A) for static 0x44EE76D5 -> ~/builds/my_led/overlay/my_led
imported into the store (...): it shows in Program
```

You have changed FPGA logic and run it on the board. The same flow builds a whole nanoSoC with your own accelerator on its expansion port: User Guide chapter 7.

15 When something goes wrong

The problems a new owner is most likely to meet, then a recovery ladder for a board that has stopped making sense.

YOU SEE	CAUSE, AND WHAT TO DO
The board does not answer: the window cannot open it, or <code>info</code> says <code>offline</code>	The board does not answer. Wait 3 to 4 minutes after any power-on. Then check the power, the cable, your PC's address (6.4), and that no other network on your PC uses <code>192.168.10.x</code> . A firewall on your PC can also drop the board's replies: allow traffic from the board's address.
Windows: <code>info</code> answers, but the window finds no board, or the claim times out	Windows Firewall drops the board's UDP replies. Make the board's network Private, add the UDP rule, and disable any Python block rule (6.4, step 3).
Windows: <code>probe</code> lists no serial port	Open Device Manager → Ports (COM & LPT) : it should list four USB Serial Port (COMn) for the board. If they are missing, or under Other devices , install the FTDI VCP driver (Windows Update offers it; otherwise <code>ftdichip.com</code> , VCP Drivers), then unplug and replug the Debug USB. Install Harness Manager with <code>-WithSerial</code> .
<code>info</code> says <code>rescue</code> after chapter 8	The harness found no Linux to start. Check that the microSD is in the user slot and is larger than 2 GB, then write it again (chapter 8) and claim again (chapter 9).
<code>no permission to open serial port</code> , or <code>probe</code> lists no serial port	On Linux, join the <code>dialout</code> group and log in again (6.2). Install Harness Manager with <code>--with-serial</code> .
After a restart, <code>board claim-status</code> says <code>unclaimed</code> ; or Harness Manager refuses the host key	The claim was kept only in memory, or the card is new: do 9.1 steps 2 to 4. A changed host key is expected after a new card and after 9.1's restart; anywhere else, find out why before you accept it with <code>--replace-host-key</code> .
The console shows nothing	The design printed before you connected. In the window, tick Arm and click Reset DUT . On the command line, open the console first, then reset: Ctrl-], r, r (chapter 10). Ctrl-] leaves the console; Ctrl-C does not.
<code>is in use – your own `harness-manager debug up` ... holds it</code>	On the command line, one command at a time uses the board. Press Ctrl-C in the <code>debug up</code> terminal, or Ctrl-] in an open <code>console</code> , then try again. The window runs a console and a debug session together (chapter 12).
GDB says <code>Servname not supported</code> , or your PC's own GDB starts instead of <code>arm-none-eabi-gdb</code>	The copied command was pasted twice, or after something you typed. Paste it once, alone, at an empty prompt (chapter 12, step 4).
nanoSoC prints nothing, and GDB shows <code>pc 0xffffffffe; monitor reset halt</code> prints <code>JTAG-DP STICKY ERROR</code>	The processor has locked up, and <code>monitor reset halt</code> alone does not clear it; for your own program, chapter 13 shows how to catch the processor in its boot ROM instead. The board's QSPI flash holds a boot image from an earlier MicroPython install, which the plain <code>nanosoc</code> design cannot start. If you installed MicroPython, use <code>nanosoc_upy</code> instead, which you build as you built <code>nanosoc</code> (User Guide §7.7); otherwise ask SoC Labs support.
A load or restore is refused: <code>`mps3-debug down` failed before the swap</code> (exit 15 on the command line)	Before every load, Harness Manager stops the debugger on the board, and this time it could not. Nothing was loaded. Try again; if it repeats, the window offers Program anyway (armed like Program), and the command line takes <code>--force</code> (User Guide chapter 3).

15.1 The recovery ladder

Climb from the top, and stop as soon as the board behaves.

In the window, **Board** → **Recover** has its own ladder, headed **Gentle to heavy: start at the top and stop at the first step that brings the board back**. Each step says what it keeps and loses, and needs **Arm** first: 1 **Reset DUT** (about a second), 2 **Back to greybox** (about 40 seconds), 3 **Restart the shell** (3 to 4 minutes: the harness restarts without reloading the FPGA), 4 **Reboot via the MCC** (about 3 minutes: the FPGA reloads from the config SD), and 5 **Power-cycle**, which needs a networked power plug; without one, the page says so, and you use the switch (step 3

below). The page also says which steps this board can do now. In Harness Manager 1.0.0 its own estimates for steps 2 and 3 are too low: expect the times given here.

- 1 **Ask the board.** The window's header and **Overview** say what it runs and whether it is healthy. Command line: `harness-manager info $B`; its control line (`idle`, `busy`, `offline`, `rescue`) and notes say what it thinks is wrong.
- 2 **Put greybox back: Restore baseline** in the Workbench (or `harness-manager restore $B`). This cures a design that misbehaves, as long as the harness answers.
- 3 **Switch the board off for ten seconds, then on.** Linux is back in 3 to 4 minutes, with the partition empty.
- 4 **Put the config SD back as it was, and install again.** This step is on the command line. With the Debug USB connected: `harness-manager sd - --volume $DRIVE restore ~/mps3-backups/<zip>`, then chapter 7, steps 4 to 6.
- 5 **Ask for help:** SoC Labs support (the support contact that came with your board). Send the output of `harness-manager --json info $B` and `harness-manager version`, the log `~/config/harness-manager/daemon.log`, and what you did and what it said.

User Guide chapter 9 has the full troubleshooting tables, and its appendix lists the meaning of every exit code.

16 Where next

The User Guide picks up where this guide stops, task by task. Go by what you want to do.

I WANT TO...	READ IN THE USER GUIDE
Understand what happens inside the board	Chapter 1, How the platform works
Update the harness, change my board's name or address, recover a board	Chapter 2, Your board
Load the other designs, or start the board with one	Chapter 3, Designs
Know nanoSoC's memory map, boot and variants	Chapter 4, nanoSoC in depth
Debug further, or use logic analysers inside a design	Chapter 5, Consoles and debugging
Build software into a design, or run it from flash or MicroPython	Chapter 6, Software on nanoSoC
Put my own SoC or accelerator in the partition	Chapter 7, Building your own FPGA design
Script tests from Python or the command line	Chapter 8, The command line and scripting

The Technical Reference & Maintainers Guide explains how the harness itself is built, for when you need to look underneath.

A Cheat sheet

Window paths start from the open board (**Open board**). Commands start with `harness-manager` unless they start with `arm-none-eabi-`, `(gdb)` or `make`. `$B` is your board (192.168.10.101 until you change it); `$PORT` and `$DRIVE` are the MCC's serial port and the config SD's drive (chapter 7). Add `--json` to any `harness-manager` command for machine-readable output.

TO...	IN THE WINDOW	ON THE COMMAND LINE
Once per board		
Find the port and the drive	–	<code>probe</code>
Back up the config SD	–	<code>sd - --volume \$DRIVE backup ~/mps3-backups</code>
Install the harness	–	<code>sd - --volume \$DRIVE install ~/mps3-bundle/config-sd --backup <zip></code>
Reboot through the MCC	–	<code>mcc - --serial \$PORT reboot</code>
Add the board	+ → By address → Add; Open board	(commands take the address: <code>\$B</code>)
Claim it (twice on a new card)	Board → Access → Claim with my key... (the first claim)	<code>board claim \$B --key ~/.ssh/id_ed25519.pub;</code> <code>board ssh \$B -c 'mps3-persist format --erase';</code> power-cycle; <code>board claim \$B --key ... --replace-host-key</code>
Name it	– (check it in Board → Access)	<code>board identity \$B --label NAME --mac 02:... [--ip .../24]</code>
Where your designs are	Settings → Harness & kits	<code>config set mps3.overlay_dirs ~/mps3-overlays</code>
Every session		
What is on the board; is it well?	The header; Overview	<code>info \$B</code>
Load a design	Workbench: pick it, Arm, Program	<code>program \$B nanosoc</code> (asks; <code>--yes</code> in scripts)
Back to greybox	Arm, Restore baseline	<code>restore \$B</code>
The DUT console; reset the DUT	DUT console; Arm, Reset DUT	<code>console \$B uart0 : Ctrl-] exits, Ctrl-] r r resets. Read-only: nc \$B 6930</code>
Debug	Debug → Open session; copy the GDB command and paste it alone at a prompt (<code>c</code> at GDB's pager prompt)	<code>debug up \$B</code> (Ctrl-C ends it), then its <code>attach</code> line; <code>debug status \$B</code>
In GDB	<code>(gdb) monitor halt · info registers pc sp · detach</code>	
The front panel	Overview → Front panel	–
The window itself	<code>app</code> (a demo with no board: <code>app --demo</code>)	
Build the nanoSoC design, once (chapter 11; needs Arm Academic Access)		
Build <code>nanosoc</code>	No button yet: use the command line	In the design folder: <code>kit script ... --design nanosoc.json --out ~/builds/nanosoc</code> , then as for chapter 14
Your own program (chapter 13)		
Get and build	<code>cp -r ~/mps3-platform/examples/nanosoc_hello ~/myhello · make</code>	

TO...	IN THE WINDOW	ON THE COMMAND LINE
Load and run	<code>(gdb) monitor reset halt · monitor nanosoc.cpu0 arp_examine · monitor halt · load · monitor reg pc 0x10000040 · monitor reg sp 0x18004000 · monitor reg xPSR 0x01000000 · monitor resume</code>	
Your own FPGA design (chapter 14)		
Kit, design, script	Build: Setup, Design, Build, Check, Add	<code>kit import <kit zip> · kit guide --static-id 0x44EE76D5 --design my_led.json · kit script ... --out ~/builds/my_led</code>
Build, check, add, load		<code>kit build ~/builds/my_led (prints the Vivado command) · kit check ~/builds/my_led · kit pack ~/builds/my_led --import · program \$B my_led</code>
When it goes wrong (chapter 15)		
The recovery ladder	Board → Recover	see 15.1
Put the config SD back	–	<code>sd - --volume \$DRIVE restore ~/mps3-backups/<zip></code>

B Glossary

Arm Academic Access	Arm's IP licence for universities: you need it to build the nanoSoC designs (chapter 11).
bitstream	The file that configures an FPGA's logic (1.1).
claim	Sending your SSH key to the board, which then trusts only that key (4.3, 9.1).
config SD	The MCC's card: the bitstream the board loads at power-on. Appears on your PC as <code>V2M-MPS3</code> (1.1).
Debug USB	The board's USB socket: the config SD as a drive, and four serial ports, one of them the MCC's (1.1).
design file	The <code>.json</code> that tells Harness Manager how to build a design (chapters 11, 14).
design, overlay	What you load into the partition; on disk, a directory with a partial, a clearing file and a manifest (4.3).
DUT	Device under test: the design running in the partition (2.2).
FPGA	Field-programmable gate array: a chip whose logic you configure (1.1).
greybox	The empty design; Harness Manager's <i>baseline</i> (4.3).
harness	The permanent design on the FPGA: a Linux computer that loads designs and serves the consoles and debugger (2.2).
Harness Manager	The program on your PC you drive the board from: a window, and the same as commands (4.1, 4.2).
host key	The key a board proves its identity with; Harness Manager records it at the claim (9.1).
IMEM, DMEM	nanoSoC's instruction and data memories (3.1).
kit	The build kit for one harness: what Vivado needs to build a design for its partition (chapters 11, 14).
load (program)	Writing a design into the partition, in about a minute (4.3).
MCC	The board's controller: switches it on and loads the FPGA from the config SD (1.1).
OpenOCD, GDB	The debug server, which runs on the board, and the debugger you run on your PC (6.3, chapter 12).
partition	The region of the FPGA that holds the DUT, rewritten while the harness runs (2.2).
preflight	The checks Harness Manager runs before a load (chapter 10).
rescue	The harness waiting for a Linux to start, because it found none on the user microSD (chapter 7).
rm_id	A design's own 32-bit ID, read back after every load to verify it (chapter 14).

static ID	The harness's fingerprint (Shell in the header): 0x44ee76d5 for v2.0. A design loads only on its own harness (chapter 10).
user microSD	The FPGA's card: the harness's Linux, your claim and your board's name (chapter 8).
Workbench	The window's main tab: load a design, its console, the debugger (4.1).